

Jenkins Task 1

Assignment Overview

This document covers six independent tasks: installing Jenkins and running it on a custom port (8081), securing the Jenkins server against unauthorized access, creating two users (DevOps and Testing) with limited access, configuring labels so jobs execute only on matching agents, creating three sample Freestyle jobs against a public GitHub repository, and creating one Freestyle job against a private GitHub repository using stored credentials. Each task resets and is documented with its own independent steps.

The following resource reference is used throughout this document:

Task	Resource	Purpose
1	Jenkins server (port 8081)	Install and run the CI/CD automation server
2	Jenkins Global Security configuration	Prevent unauthorized access to Jenkins
3	Jenkins users: DevOps, Testing	Provide limited-access accounts for job execution
4	Node labels + job restriction	Restrict jobs to execute only on designated agents
5	Three Freestyle jobs (public repo)	Build automation from a public GitHub repository
6	One Freestyle job (private repo)	Build automation from a private GitHub repository using credentials

Task 1: Install Jenkins and Run Jenkins on Port 8081

Outcome

Jenkins is installed on the target server and the Jenkins service is reachable in the browser at `http://<server-ip>:8081`, instead of the default port 8080.

Objective

To stand up a working Jenkins automation server on a non-default port, so it does not collide with other services already using 8080 and is ready for further security and job configuration.

Prerequisites

- A Linux server (Amazon Linux/RHEL/Ubuntu) with sudo access and internet connectivity.
- Java (JDK 17 or 21) installed, since Jenkins runs on the JVM.
- Inbound security group / firewall rule allowing TCP port 8081.

Step-by-Step Implementation

Step 1: Install Java

```
sudo yum install java-21-amazon-corretto -y
```

 **Screenshot Evidence:** capture the console view for this step and insert it here.

```
Verifying      : libxcb-1.17.0-1.amzn2023.0.1.x86_64
Verifying      : pixman-0.43.4-1.amzn2023.0.4.x86_64
Verifying      : xml-common-0.6.3-56.amzn2023.0.2.noarch

Installed:
alsa-lib-1.2.7.2-1.amzn2023.0.3.x86_64
dejavu-sans-fonts-2.37-16.amzn2023.0.2.noarch
dejavu-serif-fonts-2.37-16.amzn2023.0.2.noarch
fonts-filesystem-1:2.0.5-12.amzn2023.0.2.noarch
glib-5.2.1-9.amzn2023.0.4.x86_64
google-noto-sans-vf-fonts-20240401-1.amzn2023.0.2.noarch
harfbuzz-7.0.0-2.amzn2023.0.2.x86_64
java-21-amazon-corretto-headless-1:21.0.12+8-1.amzn2023.1.x86_64
langpacks-core-font-en-3.0-21.amzn2023.0.4.noarch
libSM-1.2.4-3.amzn2023.0.1.x86_64
libX11-common-1.8.10-2.amzn2023.0.1.noarch
libXext-1.3.6-1.amzn2023.0.1.x86_64
libXinerama-1.1.5-6.amzn2023.0.1.x86_64
libXrender-0.9.11-6.amzn2023.0.1.x86_64
libXtst-1.2.5-1.amzn2023.0.1.x86_64
libjpeg-turbo-2.1.4-2.amzn2023.0.5.x86_64
libxcb-1.17.0-1.amzn2023.0.1.x86_64
xml-common-0.6.3-56.amzn2023.0.2.noarch

cairo-1.18.0-4.amzn2023.0.3.x86_64
dejavu-sans-mono-fonts-2.37-16.amzn2023.0.2.noarch
fontconfig-2.13.94-2.amzn2023.0.2.x86_64
freetype-2.13.2-5.amzn2023.0.2.x86_64
google-noto-fonts-common-20240401-1.amzn2023.0.2.noarch
graphite2-1.3.14-7.amzn2023.0.3.x86_64
java-21-amazon-corretto-1:21.0.12+8-1.amzn2023.1.x86_64
javapackages-filesystem-6.0.0-7.amzn2023.0.6.noarch
libICE-1.1.1-3.amzn2023.0.1.x86_64
libX11-1.8.10-2.amzn2023.0.1.x86_64
libXau-1.0.11-6.amzn2023.0.1.x86_64
libXi-1.8.2-1.amzn2023.0.1.x86_64
libXrandr-1.5.4-3.amzn2023.0.1.x86_64
libXt-1.3.0-3.amzn2023.0.1.x86_64
libbrotli-1.0.9-4.amzn2023.0.2.x86_64
libpng-2:1.6.37-10.amzn2023.0.13.x86_64
pixman-0.43.4-1.amzn2023.0.4.x86_64

Complete!
[ec2-user@ip-172-31-17-219 ~]$ java -version
openjdk version "21.0.12" 2026-07-21 LTS
OpenJDK Runtime Environment Corretto-21.0.12.8.1 (build 21.0.12+8-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.12.8.1 (build 21.0.12+8-LTS, mixed mode, sharing)
[ec2-user@ip-172-31-17-219 ~]$
```

Step 2: Add the Jenkins Repository and Install Jenkins

Parameter	Value
Add repo	sudo wget -O /etc/yum.repos.d/jenkins.repo https://pkg.jenkins.io/redhat-stable/jenkins.repo
Install command	sudo yum install jenkins -y

 **Screenshot Evidence:** capture the console view for this step and insert it here.

```
[ec2-user@ip-172-31-17-219 ~]$ sudo wget -O /etc/yum.repos.d/jenkins.repo \
https://pkg.jenkins.io/rpm-stable/jenkins.repo
--2026-08-10 13:33:26-- https://pkg.jenkins.io/rpm-stable/jenkins.repo
Resolving pkg.jenkins.io (pkg.jenkins.io)... 146.75.38.133, 2a04:4e42:95::645
Connecting to pkg.jenkins.io (pkg.jenkins.io)|146.75.38.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 267 [application/octet-stream]
Saving to: '/etc/yum.repos.d/jenkins.repo'

/etc/yum.repos.d/jenkins.repo      100%[=====>]      267  --.-KB/s   in 0s

2026-08-10 13:33:25 (10.6 MB/s) - '/etc/yum.repos.d/jenkins.repo' saved [267/267]
```

```
[ec2-user@ip-172-31-17-219 ~]$ sudo dnf install jenkins
Jenkins-stable                               11 kB/s | 833 B   00:00
Jenkins-stable                               98 kB/s | 1.6 kB  00:00
Importing GPG key 0x14ABFC68:
  Userid      : "Jenkins Project <jenkinsci-board@googlegroups.com>"
  Fingerprint: 5E38 6EAD B55F 0150 4CAE 8BCF 7198 F4B7 14AB FC68
  From        : https://pkg.jenkins.io/rpm-stable/repodata/repomd.xml.key
Is this ok [y/N]: y
Jenkins-stable                               37 kB/s | 2.6 kB  00:00
Dependencies resolved.

Package Architecture Version Repository Size
Installing:
jenkins      noarch      2.568.2-1 jenkins    96 M

Transaction Summary
Install 1 Package

Total download size: 96 M
Installed size: 96 M
Is this ok [y/N]: y
Downloading Packages:
jenkins-2.568.2-1.noarch.rpm                  77 MB/s | 96 MB   00:01
Total
Jenkins-stable                               76 MB/s | 96 MB   00:01
Importing GPG key 0x14ABFC68:
  29 kB/s | 1.6 kB  00:00
```

```
Total
Jenkins-stable
Importing GPG key 0x14ABFC68:
 76 MB/s | 96 MB 00:01
 29 kB/s | 1.6 kB 00:00
  Userid   : "Jenkins Project <jenkinsci-board@googlegroups.com>"
  Fingerprint: 5E38 6EAD B55F 0150 4CAE 8BCF 7198 F4B7 14AB FC68
  From      : https://pkg.jenkins.io/rpm-stable/repodata/repomd.xml.key
Is this ok [y/N]: y
Key imported successfully
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing      :
  Running scriptlet: jenkins-2.568.2-1.noarch 1/1
  Installing     : jenkins-2.568.2-1.noarch 1/1
  Running scriptlet: jenkins-2.568.2-1.noarch 1/1
  Verifying      : jenkins-2.568.2-1.noarch 1/1

Installed:
jenkins-2.568.2-1.noarch

Complete!
[ec2-user@ip-172-31-17-219 ~]$
```

Step 3: Change the Default Port to 8081

Edit the Jenkins service configuration so it listens on 8081 instead of 8080:

Parameter	Value
Config file	/usr/lib/systemd/system/jenkins.service (or /etc/sysconfig/jenkins)
Parameter changed	Environment="JENKINS_PORT=8080"
New value	Port=8081

sudo systemctl daemon-reload

 **Snapshot Evidence:** capture the console view for this step and insert it here.

```
# # Port to listen on for HTTP requests. Set to -1 to disable.
# # To be able to listen on privileged ports (port numbers less than 1024),
# # add the CAP_NET_BIND_SERVICE capability to the AmbientCapabilities
# # directive below.
# Environment="JENKINS_PORT=8081"
#
# # IP address to listen on for HTTPS requests. Default is disabled.
# #Environment="JENKINS_HTTPS_LISTEN_ADDRESS="
#
# # Port to listen on for HTTPS requests. Default is disabled.
# # To be able to listen on privileged ports (port numbers less than 1024),
# # add the CAP_NET_BIND_SERVICE capability to the AmbientCapabilities
# # directive below.
```

Step 4: Start and Enable Jenkins

Parameter	Value
Start service	sudo systemctl start jenkins
Enable on boot	sudo systemctl enable jenkins
Check status	sudo systemctl status jenkins

 **Snapshot Evidence:** capture the console view for this step and insert it here.

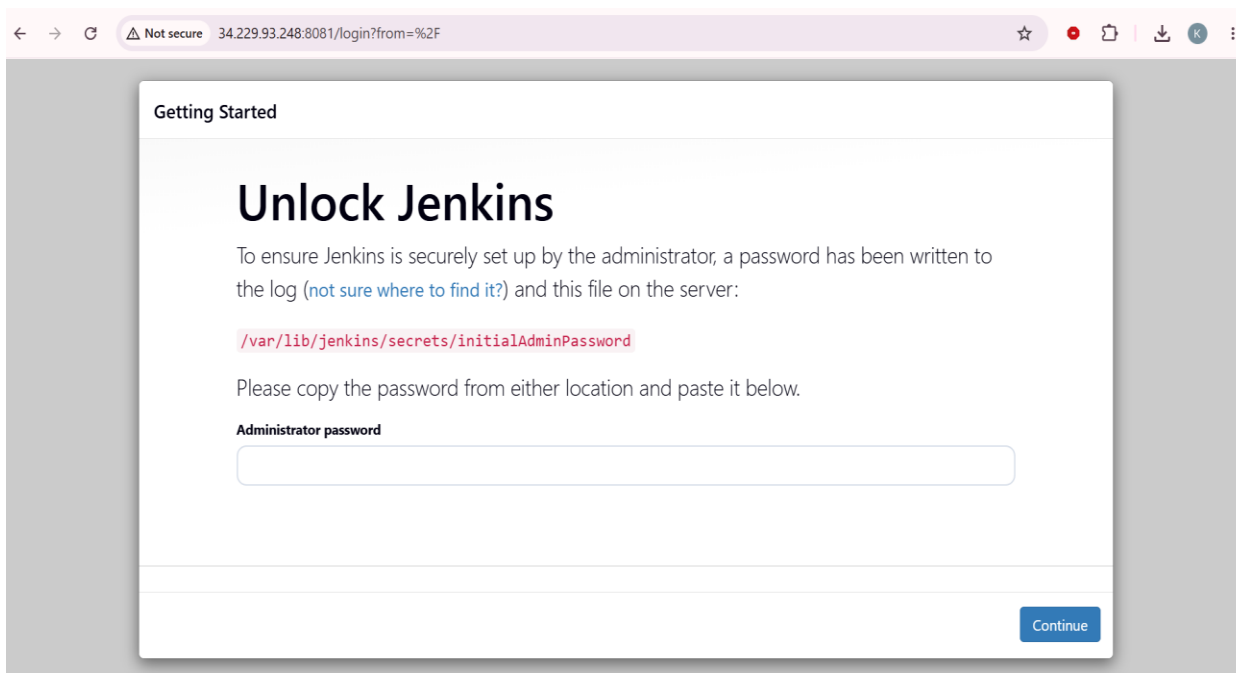
```
[root@ip-172-31-17-219 ~]# sudo systemctl status jenkins
jenkins.service - Jenkins Continuous Integration Server
Loaded: loaded (/usr/lib/systemd/system/jenkins.service; disabled; preset: disabled)
Active: active (running) since Mon 2026-08-10 13:45:57 UTC; 17s ago
Main PID: 27254 (java)
Tasks: 42 (limit: 1059)
Memory: 392.5M
CPU: 19.015s
CGroup: /system.slice/jenkins.service
└─27254 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war --httpPort=8080

Aug 10 13:45:53 ip-172-31-17-219.ec2.internal jenkins[27254]: [LF]>
Aug 10 13:45:53 ip-172-31-17-219.ec2.internal jenkins[27254]: [LF]> *****
Aug 10 13:45:53 ip-172-31-17-219.ec2.internal jenkins[27254]: [LF]> *****
Aug 10 13:45:53 ip-172-31-17-219.ec2.internal jenkins[27254]: [LF]> *****
Aug 10 13:45:57 ip-172-31-17-219.ec2.internal jenkins[27254]: 2026-08-10 13:45:57.709+0000 [id=32] INFO jenkins.InitReactorRunner$1#onAttained: Completed initial
Aug 10 13:45:57 ip-172-31-17-219.ec2.internal jenkins[27254]: 2026-08-10 13:45:57.752+0000 [id=24] INFO hudson.lifecycle.Lifecycle#onReady: Jenkins is fully up a
Aug 10 13:45:57 ip-172-31-17-219.ec2.internal systemd[1]: Started jenkins.service - Jenkins Continuous Integration Server.
Aug 10 13:45:57 ip-172-31-17-219.ec2.internal jenkins[27254]: 2026-08-10 13:45:57.958+0000 [id=50] INFO h.m.DownloadService$Downloadable#load: Obtained the updat
Aug 10 13:45:57 ip-172-31-17-219.ec2.internal jenkins[27254]: 2026-08-10 13:45:57.958+0000 [id=50] INFO hudson.util.Retrier#start: Performed the action check upo
Aug 10 13:46:02 ip-172-31-17-219.ec2.internal jenkins[27254]: 2026-08-10 13:46:02.822+0000 [id=70] WARNING h.n.DiskSpaceMonitorDescriptor#markNodeOfflineOrOnline
```

Step 5: Unlock Jenkins and Complete Setup Wizard

Parameter	Value
URL	http://<server-ip>:8081
Initial admin password file	sudo cat /var/lib/jenkins/secrets/initialAdminPassword
Plugins	Install suggested plugins

 **Screenshot Evidence:** capture the console view for this step and insert it here.



Validation Steps

- sudo systemctl status jenkins shows "active (running)".
- http://<server-ip>:8081 loads the Jenkins login/unlock screen in the browser.
- sudo ss -tlnp | grep 8081 shows Jenkins listening on the port.

 **Screenshot Evidence:** capture the console view for this step and insert it here.

```
[root@ip-172-31-17-219 system]# sudo ss -lntp | grep 8080
[root@ip-172-31-17-219 system]# sudo ss -lntp | grep 8081
LISTEN 0      50          *:8081      *:~      users: ( ("java",pid=28019,fd=9) )
[root@ip-172-31-17-219 system]#
```

Explanation

Jenkins ships with a default HTTP port of 8080, which frequently conflicts with other web applications or Tomcat instances running on the same host. Overriding `--httpPort` at the service level moves Jenkins to 8081 without changing any other behavior, and `systemctl daemon-reload` ensures `systemd` picks up the modified unit definition before the service is (re)started.

Real-Time Use Case

Organizations frequently run several Java-based servers (Tomcat, application servers, other CI tools) on the same host or the same subnet, so pinning Jenkins to a dedicated, documented port such as 8081 avoids port collisions and makes firewall/security-group rules easier to standardize across environments.

Common Errors and Troubleshooting

Error	Cause	Resolution
Jenkins fails to start after port change	Syntax error in the service/config file, or daemon-reload skipped	Verify the <code>--httpPort=</code> value and run <code>sudo systemctl daemon-reload</code> before restarting
Page not loading at :8081	Security group / firewall does not allow inbound TCP 8081	Add an inbound rule for port 8081 to the instance's security group
initialAdminPassword file not found	Jenkins has not finished its first startup yet	Wait for the service to report active (running), then re-check the secrets path

Task 2: Secure the Jenkins Server

Outcome

Jenkins is configured with a dedicated administrator account, anonymous read access is disabled, and Role-Based Authorization Strategy is enabled so that all further access is controlled explicitly rather than left open by default.

Objective

To close the default open-access posture of a freshly installed Jenkins server, ensuring only authenticated, explicitly authorized users can view or operate jobs.

Prerequisites

- Task 1 completed (Jenkins installed and reachable on port 8081).
- Role-based Authorization Strategy plugin installed (via Manage Plugins).

Step-by-Step Implementation

Step 1: Install the Project-based Matrix Authorization Strategy

Manage Jenkins -> Plugins -> Available plugins -> search "Role-based Authorization Strategy" -> Install without restart.

 **Screenshot Evidence:** *capture the console view for this step and insert it here.*

Matrix Authorization Strategy	✓ Success
LDAP	✓ Success
jsoup API	✓ Success
Email Extension	✓ Success
Mailer	✓ Success
Theme Manager	✓ Success
Dark Theme	✓ Success
Loading plugin extensions	✓ Success
Role-based Authorization Strategy	✓ Success
Loading plugin extensions	✓ Success

→ [Go back to the top page](#)
 (you can start using the installed plugins right away)

→ ☐ Restart Jenkins when installation is complete and no jobs are running

Step 2: Enable Security

Parameter	Value
Location	Manage Jenkins -> Security -> Configure Global Security
Security Realm	Jenkins' own user database
Allow users to sign up	Disabled
Authorization	Project-based Matrix Authorization Strategy

 **Screenshot Evidence:** capture the console view for this step and insert it here.

☐ Allow users to sign up ?

Authorization

Project-based Matrix Authorization Strategy

Add user

Add group

Anonymous No permissions granted	
Authenticated Users No permissions granted	

Markup Formatter

Markup Formatter ?

Save

Apply

Explanation

A default Jenkins installation with security left unconfigured, or with the Anonymous user granted read access, exposes job configuration, build logs, and credentials to anyone who can reach the URL. Enabling Jenkins' own user database plus Role-Based Authorization Strategy moves Jenkins from an implicit-allow model to an explicit-allow model, where every permission must be granted deliberately to a role.

Real-Time Use Case

In shared or internet-reachable environments, an unsecured Jenkins instance is a common attack vector because it can execute arbitrary shell commands as part of a build; locking it down immediately after installation is treated as a mandatory hardening step before any real project is onboarded.

Common Errors and Troubleshooting

Error	Cause	Resolution
Locked out of Jenkins after enabling security	No administrator role was assigned before saving	Edit <code>/var/lib/jenkins/config.xml</code> on the server to temporarily disable security, then reconfigure roles correctly
Project-based Matrix Authorization Strategy	Plugin not installed or Jenkins not restarted after install	Install the Project-based Matrix Authorization Strategy plugin and refresh the Configure Global Security page

Task 3: Create Users DevOps and Testing with Limited Access

Outcome

Two Jenkins user accounts, DevOps and Testing, exist in the Jenkins user database, each assigned a role that grants only the permissions appropriate to their function (build/view access) rather than full administrator rights.

Objective

To apply least-privilege access control so day-to-day operators can run and monitor jobs without being able to change global Jenkins configuration, security settings, or unrelated jobs.

Prerequisites

- Task 2 completed (Role-Based Authorization Strategy enabled).
- Administrator access to Jenkins.

Step-by-Step Implementation

Step 1: Create the User Accounts

Parameter	Value
Location	Manage Jenkins -> Users -> Create User
User 1	Username: DevOps
User 2	Username: Testing
Password	Set a temporary password for each; instruct users to change it after first login

 **Screenshot Evidence:** capture the console view for this step and insert it here.

Users 3

+ Create User

Username 1	Full name	
 admin	admin	
 devops	Devops123	 
 testing	Testing	 

Step 2: Assign Roles to Users

Manage Jenkins -> Security -> Project-based Matrix Authorization Strategy -> add DevOps , and Testing.

 **Screenshot Evidence:** capture the console view for this step and insert it here.

Project-based Matrix Authorization Strategy

Filter users and groups

Add user

Add group

admin Overall: Administer

Devops123 Overall: Read · View: Read

Testing Overall: Manage · Job: Build

Anonymous No permissions granted

Authenticated Users No permissions granted

Markus Formetter

Validation Steps

- Logging in as DevOps shows only the permitted jobs and allows triggering builds, but Manage Jenkins is not accessible.
- Logging in as Testing shows the same jobs in read/build-only mode with no configuration options visible.

Explanation

Role-Based Authorization Strategy separates permissions into global roles (instance-wide) and project (item) roles that apply only to jobs matching a name pattern. Granting DevOps and Testing project roles instead of global roles means their access is confined to the specific jobs they need, and any new unrelated job created later is not automatically visible to them.

Real-Time Use Case

In real projects, developers typically need to trigger and inspect their own build/deploy jobs, while a QA/testing team only needs to view build and test results; separating these roles prevents accidental configuration changes and keeps an audit trail of who is permitted to do what.

Common Errors and Troubleshooting

Error	Cause	Resolution
User can see all jobs, not just assigned ones	Project role pattern was too broad (e.g. .*)	Tighten the regular expression pattern to match only the intended job names
User cannot build even after role assignment	Build permission not checked in the item role	Edit the role under Manage Roles and enable Job/Build
New user cannot log in	Password not set correctly, or account created without saving	Reset the password from Manage Jenkins -> Users and confirm the account is saved

Task 4: Configure Labels and Restrict Jobs to Execute Based on Label

Outcome

A label is assigned to the target Jenkins node/agent, and jobs are configured with "Restrict where this project can run" set to that label, so a job only executes on an agent carrying the matching label instead of any available executor.

Objective

To control which agent(s) a job is allowed to run on, which is important when different jobs require different tools, operating systems, or capacity, or when certain agents are reserved for specific teams or environments.

Prerequisites

- At least one Jenkins node/agent configured (the built-in controller node can also carry a label for this exercise).
- Administrator access to Jenkins.

Step-by-Step Implementation

Step 1: Assign a Label to the Node

Parameter	Value
-----------	-------

Location	Manage Jenkins -> Nodes -> select node -> Configure
Labels field	linux-build
Usage	Only build jobs with label expressions matching this node

 **Screenshot Evidence:** capture the console view for this step and insert it here.

Number of executors 

2

Labels 

linux-build

Usage 

Only build jobs with label expressions matching this node

Node Properties

- ☐ Disable deferred wipeout on this node 
- ☐ Disk Space Monitoring Thresholds
- ☐ Enable node-based security

Step 2: Restrict a Job to the Label

Parameter	Value
Location	Job -> Configure -> General
Option	Restrict where this project can run
Label Expression	linux-build

 **Screenshot Evidence:** capture the console view for this step and insert it here.

- ☐ Execute concurrent builds if necessary 
- ☒ Restrict where this project can be run 

Label Expression 

linux-build

Label linux-build matches 1 node. Permissions or other restrictions provided by plugins may further reduce that list.

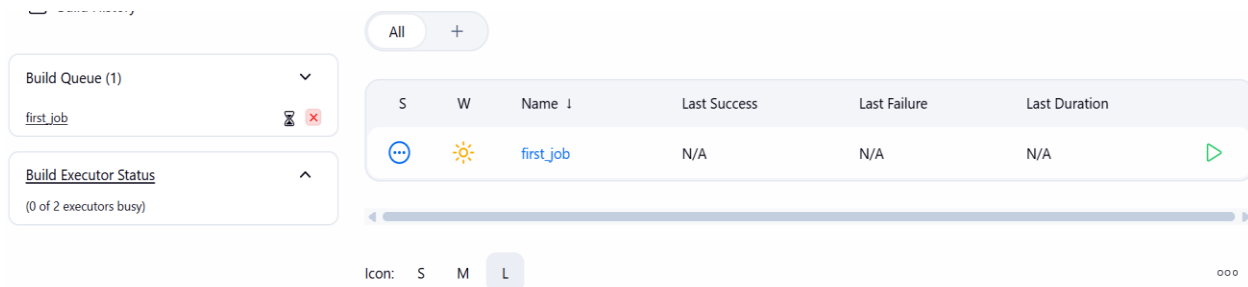
Advanced 

Validation Steps

- Triggering the restricted job schedules the build only on the node carrying the matching label (visible in the build's console output / node column).

- Temporarily removing the label from the node causes the job to sit in the queue with a "waiting for next available executor" message.

Screenshot Evidence: [capture the console view for this step and insert it here.](#)



Explanation

Jenkins schedules a job onto any connected executor by default. Setting a label expression on a job changes this to a constraint: Jenkins will only place that build on a node whose own labels satisfy the expression, which can be a single label, multiple labels combined with AND/OR/NOT operators, or a built-in name.

Real-Time Use Case

Teams commonly use labels to separate build environments, for example pinning jobs that need Docker to docker-enabled agents, Windows-specific builds to windows agents, or high-memory jobs to a dedicated high-mem pool, ensuring builds always land on hardware/software that actually meets their requirements.

Common Errors and Troubleshooting

Error	Cause	Resolution
Job stuck in queue indefinitely	No connected node carries the required label	Verify the label spelling matches exactly on both the node and the job, and confirm the node is online
Job still runs on the wrong node	Restrict where this project can run left blank	Re-open the job configuration and confirm the label expression was saved
Label expression rejected as invalid	Unsupported syntax (typo in AND/OR/NOT operators)	Use the built-in label expression validator link shown under the field to check syntax

Task 5: Create Three Sample Jobs from the Public GitHub Repository

Outcome

Three independent Freestyle jobs exist in Jenkins, each configured to pull source code from https://github.com/betawins/Techie_horizon_Login_app.git and each executes successfully, confirming Jenkins can check out and build from a public Git repository under different job configurations.

Objective

To demonstrate repeatable Git-based job creation in Jenkins and confirm that source-control integration, build triggers, and workspace checkout all function correctly against a real public repository.

Prerequisites

- Git plugin installed in Jenkins (installed by default with the suggested plugin set).
- Git client installed on the Jenkins node (git --version).
- Outbound internet access from the Jenkins server to github.com.

Step-by-Step Implementation

Step 1: Create Job 1 (Freestyle Project)

Parameter	Value
Location	Dashboard -> New Item
Item name	techie-horizon-job-1
Type	Freestyle project
Source Code Management	Git
Repository URL	https://github.com/betawins/Techie_horizon_Login_app.git
Branch	*/main

 **Screenshot Evidence:** capture the console view for this step and insert it here.

New Item

Enter an item name

techie-horizon-job-1

Select an item type



Pipeline

Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.



Freestyle project

Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.



Multi-configuration project

Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

OK

Git ?

Repositories ?

Repository URL ?

`https://github.com/betawins/Techie_horizon_Login_app.git`

Credentials ?

- none -

+ Add

Advanced ▾

+ Add Repository

Branches to build ?

Branch Specifier (blank for 'any') ?

`*/master`

Save Apply

Step 2: Create Job 2 (Freestyle Project with Poll SCM Trigger)

Parameter	Value
Item name	techie-horizon-job-2
Type	Freestyle project
Repository URL	<code>https://github.com/betawins/Techie_horizon_Login_app.git</code>
Build Trigger	Poll SCM: H/5 * * * *

 **Screenshot Evidence:** capture the console view for this step and insert it here.

New Item

Enter an item name

techie-horizon-job-2

Select an item type



Pipeline

Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.



Freestyle project

Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.



Multi-configuration project

Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.



Folder

Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.



Multibranch Pipeline

Creates a set of Pipeline projects according to detected branches in one SCM repository.

OK

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

☐ Trigger builds remotely (e.g., from scripts) ?

☐ Build after other projects are built ?

☐ Build periodically ?

☐ GitHub hook trigger for GITScm polling ?

☒ Poll SCM ?

Schedule ?

Poll SCM: H/5 * * * *

No schedules so will only run due to SCM changes if triggered by a post-commit hook

☐ Ignore post-commit hooks ?

Save

Apply

Step 3: Create Job 3 (Freestyle Project with Build Step)

Parameter	Value
Item name	techie-horizon-job-3
Type	Freestyle project
Repository URL	https://github.com/betawins/Techie_horizon_Login_app.git
Build Step	Execute shell: mvn clean package (or the project's applicable build command)

 **Screenshot Evidence:** capture the console view for this step and insert it here.

New Item

Enter an item name

Select an item type



Pipeline

Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.



Freestyle project

Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.



Multi-configuration project

Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.



Folder

Creates a container that stores nested items in it. Useful for grouping things together. Unlike view,

OK

Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment.

Execute shell ?

Command

See the list of available environment variables

```
echo "Java Version:"
java -version
```

Advanced ▾

+ Add build step

SaveApply

Step 4: Run Each Job

Save each job configuration, then select Build Now on all three jobs and confirm each completes with a status of SUCCESS in Build History.

 **Screenshot Evidence:** capture the console view for this step and insert it here.

Jenkins

+ New Item

Build History

Build Queue

No builds in the queue.

Build Executor Status

(0 of 2 executors busy)

All +

S	W	Name 1	Last Success	Last Failure	Last Duration
✓	☀	techie-horizon-job-1	9 min 18 sec #1	N/A	0.57 sec ▶
✓	☀	techie-horizon-job-2	9 min 38 sec #1	N/A	5.5 sec ▶
✓	☁	techie-horizon-job-3	5.5 sec #3	9 min 16 sec #1	0.73 sec ▶

Icon: S M L

Search

Settings

User

Add description

Validation Steps

- Each job's Build History shows a blue/green (success) build indicator.
- The workspace for each job (Job -> Workspace) contains the checked-out repository files.
- Console Output for each build shows a successful git clone/checkout of the repository.

Explanation

Each Freestyle job is an independent configuration unit in Jenkins; pointing multiple jobs at the same Git repository URL demonstrates that source checkout is reusable across jobs while build triggers, build steps, and

post-build actions can still be varied independently per job to serve different purposes (manual build, polling-based build, and a build with an explicit build step).

Real-Time Use Case

Teams commonly maintain several jobs against the same repository for different purposes, such as a manual verification job, a scheduled/polling nightly build, and a job that runs the actual compile/package step, which mirrors this task's three-job structure.

Common Errors and Troubleshooting

Error	Cause	Resolution
Checkout fails with "Could not find git repository"	Repository URL mistyped or not reachable	Re-verify the exact repository URL and test connectivity with <code>git ls-remote</code> from the Jenkins node
Build stays in queue and never starts	No executor available on the controller/node	Increase the number of executors under Manage Jenkins -> Nodes, or free a busy executor
Poll SCM job never triggers a build	Cron-style polling syntax incorrect	Correct the schedule syntax (e.g. <code>H/5 * * * *</code>) and save the job

Task 6: Create One Jenkins Job Using a GitHub Private Repository

Outcome

A Freestyle job is created and successfully checks out source code from a private GitHub repository, using a Jenkins credential (Personal Access Token or SSH key) stored securely in the Jenkins Credentials store rather than embedded in the job configuration.

Objective

To demonstrate secure integration with a private repository, where anonymous/public clone access is not possible and Jenkins must authenticate as an authorized GitHub identity.

Prerequisites

- A private GitHub repository the Jenkins server is authorized to access.
- A GitHub Personal Access Token (repo scope) or a deploy SSH key for the repository.
- Git plugin installed in Jenkins.

Step-by-Step Implementation

Step 1: Add Credentials to Jenkins

Parameter	Value
Location	Manage Jenkins -> Credentials -> System -> Global credentials -> Add Credentials
Kind	Username with password (GitHub username + Personal Access Token)
Scope	Global
ID	github-private-repo

 **Screenshot Evidence:** capture the console view for this step and insert it here.

New fine-grained personal access token

Create a fine-grained, repository-scoped token suitable for personal API use and for using Git over HTTPS.

Token name *

Jenkins Private Repo

✔ 'Jenkins Private Repo' is available.

A unique name for this token. May be visible to resource owners or users with possession of the token.

Description

Resource owner

 khadeer329

The token will only be able to make changes to resources owned by the selected resource owner. Tokens can always read all public repositories.

Expiration

 30 days (Sep 09, 2026)

The token will expire on the selected date

Only select repositories

Select at least one repository. Max 50 repositories. Also includes public repositories (read-only).

 Select repositories 1

Selected 1 repository.

 khadeer329/khadeer329-privateRepo

×

Permissions

Choose the minimal permissions necessary for your needs. [Learn more about permissions.](#)

Repositories 2

Account 0

+ Add permissions

Contents

Repository contents, commits, branches, downloads, releases, and merges. [Learn more.](#)

Access: Read and write

×

Metadata Required

Search repositories, list collaborators, and access repository metadata. [Learn more.](#)

Access: Read-only

🔒

Generate token

Cancel



Add Username with password



Global (Jenkins, nodes, items, all child items, etc) 

Username

khadeer329

☐

Treat username as secret 

Password

.....

ID 

github-private-repo

Description 

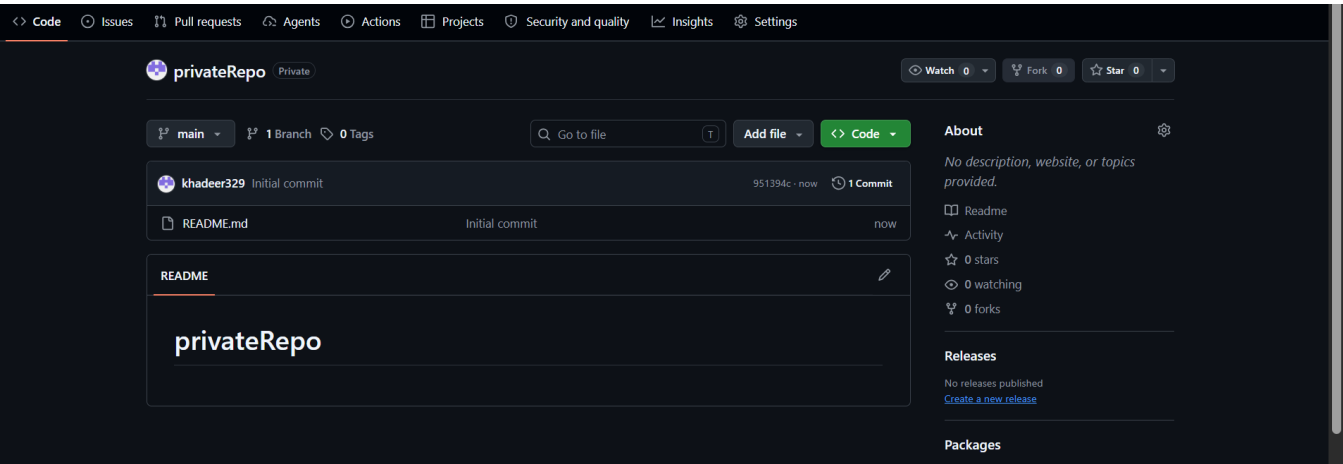
Create

Step 2: Create the Freestyle Job

Parameter	Value
Item name	techie-horizon-private-job
Type	Freestyle project

Source Code Management	Git
Repository URL	https://github.com/<org>/<private-repo-name>.git
Credentials	github-private-repo-creds (created in Step 1)
Branch	*/main

 **Screenshot Evidence:** capture the console view for this step and insert it here.



New Item

Enter an item name

Select an item type

- **Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
- **Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- **Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
- **Folder**
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view,

OK

☐ None
☒ Git ?

Repositories ?

Repository URL ?

Credentials ?

▼
+ Add

Advanced ▼

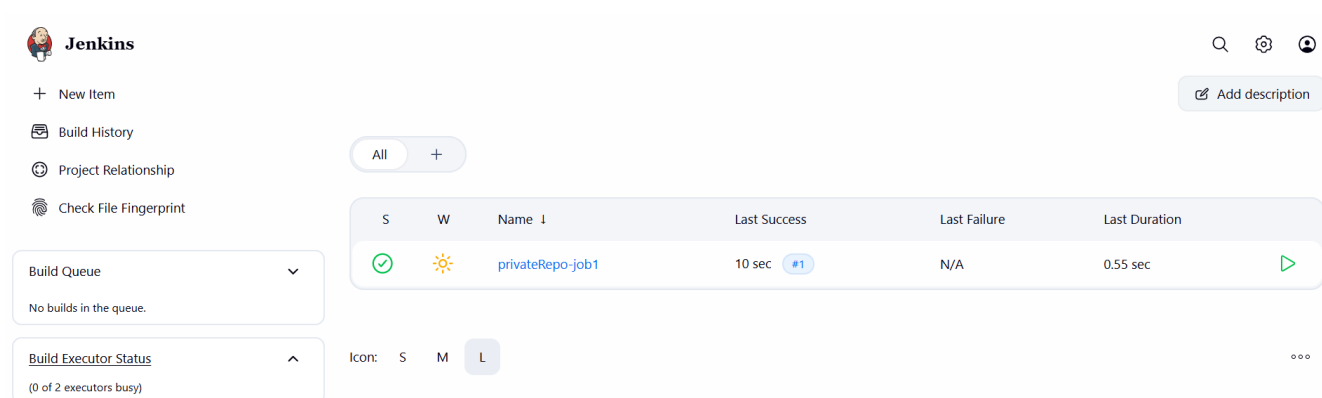
+ Add Repository

Save
Apply

Step 3: Build and Verify Checkout

Select Build Now and confirm in Console Output that the credential was used to authenticate and the repository was cloned successfully.

 **Screenshot Evidence:** *capture the console view for this step and insert it here.*



The screenshot shows the Jenkins dashboard with the following details:

- Build Queue:** No builds in the queue.
- Build Executor Status:** (0 of 2 executors busy)
- Build History Table:**

S	W	Name ↓	Last Success	Last Failure	Last Duration
✓	☀	privateRepo-job1	10 sec #1	N/A	0.55 sec
- Build Details:** Icon: S M L

Validation Steps

- The build completes with status SUCCESS instead of failing on authentication.
- Console Output shows the Git checkout using the stored credential ID, with no plaintext token/password printed.

Explanation

Private repositories reject anonymous clone requests, so Jenkins must present valid authentication. Storing the Personal Access Token inside Jenkins' Credentials Provider (rather than pasting it directly into the repository URL or job configuration) keeps the secret encrypted at rest and out of build logs, while still letting the job reference it by credential ID.

Real-Time Use Case

Most production source code lives in private repositories; this credential-based pattern is the standard way CI systems authenticate to source control, and the same approach extends to private Docker registries, artifact repositories, and cloud provider APIs used later in a pipeline.

Common Errors and Troubleshooting

Error	Cause	Resolution
"remote: Repository not found" during checkout	Wrong repository URL or credential does not have access to that repo	Confirm the URL and that the token's associated account has at least read access to the repository
Authentication failed / 403 error	Personal Access Token expired or missing required scope	Regenerate the token with the repo scope and update the stored credential
Credential not listed in job dropdown	Credential created at the wrong scope (e.g. a folder instead of Global)	Re-create the credential under Global credentials so it is visible to all jobs